# Solving Data Contamination in DDoS Detection: A Method Based on Hierarchical Federated Learning

Jiaping Gui, *Member, IEEE*, Ruiwen Ji, Haishi Huang, Jianan Hong, *Member, IEEE*, and Cunqing Hua, *Member, IEEE*

***Abstract*—Distributed Denial-of-Service (DDoS) attacks can cause significant damage to network applications. A crucial step in combating these attacks lies in promptly and accurately detecting DDoS attack traffic. However, existing solutions struggle with data imbalance and contamination, leading to suboptimal DDoS detection. Furthermore, current methods typically require access to raw data for training, posing a significant privacy risk. To tackle these challenges, we propose HFL-AD, a hierarchical federated learning framework specifically designed for detecting DDoS attack traffic by resolving the data contamination issue. In our approach, a federation of lower-layer clients train local anomaly detection models using diverse raw data. A selected few clients, possessing a small supplementary dataset, serve as upper-layer clients, responsible for excluding model updates uploaded by lower-layer clients that have been trained on contaminated datasets. Experimental results demonstrate that HFL-AD outperforms state-of-the-art (SOTA) solutions in DDoS detection, particularly when some training datasets are contaminated.**

***Index Terms*—DDoS attack, anomaly detection, federated learning, data contamination.**

## I. Introduction

DISTRIBUTED Denial-of-Service (DDoS) attacks have been a significant threat to our network. In such attacks, the attacker controls a large number of devices located in different areas, particularly the Internet of Things (IoT) devices with limited security resources [1]. These devices, under attacker control, flood the victim with traffic simultaneously. This depletes the victim's bandwidth, CPU, or memory resources. The massive traffic generated by DDoS attacks can severely impact or even disrupt the service availability of the victim host. For instance, a DDoS attack on Github in 2018 [2] reached a peak traffic of 1.35Tbps, temporarily rendering Github's services unavailable.

Due to the rapid advancement of deep learning techniques, various machine learning (ML)-based anomaly detection methods (e.g., DoS/DDoS detection [45]) have been employed to scrutinize network traffic at different levels of granularity, such as packets [3] or flows [4]. In ML-based anomaly detection, supervised learning often encounters challenges in obtaining attack traffic. Moreover, inspecting and labeling every training sample requires extensive laborforce. Hence, unsupervised learning is also applied to intrusion detection [4], [5]. A typical process of unsupervised learning involves collecting traffic data from a device's routine operations within the network. We then extract packet or flow-level features to train an anomaly detection model, assuming that all traffic is normal. Since the model learns the distribution of normal traffic during training, any traffic that deviates from this distribution will be classified as anomalous.

However, the above process encounters three practical challenges. Firstly, traffic data from a single device lacks diversity, so the model cannot learn other normal traffic patterns during training. Consequently, the model misidentifies unseen normal traffic as suspicious, thereby increasing the false positive rate. Secondly, the traffic data collected during regular operations of the device may contain a small proportion of attack traffic. Thus, the model learns a mixed distribution of normal and attack traffic, erroneously representing it as normal. This poses a data contamination problem and could have a significant impact on the model performance [6], such as a decrease in the recall rate. Unlike malicious intent that is purposeful, data contamination is often unintentional (see detailed comparison in Section III). Thirdly, collected traffic data may contain sensitive information. To protect privacy, existing ML methods cannot simply aggregate data from multiple devices for centralized training.

In this paper, we propose HFL-AD,[1] a novel approach to tackle the aforementioned three challenges. Specifically, to handle the challenges of data diversity and privacy protection, we utilize federated learning (FL) to train the anomaly detection model. In FL, participants collaboratively train a global model, thereby overcoming the issue of limited data diversity. The model can learn the joint distribution of normal traffic that is collected by all participants without disclosing sensitive data [44]. To address the challenge of data contamination, we leverage a small supplementary dataset containing labeled anomaly samples collected by a few participants. These attack traffic samples are labeled by security experts. Our insight is that participants with a supplementary dataset can exclude contaminated model updates (trained on contaminated

Received 23 December 2024; revised 14 May 2025 and 9 June 2025; accepted 2 July 2025. Date of current version 14 July 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2022YFB2702302 and in part by the Natural Science Foundation of Shanghai under Grant 23ZR1429600. The associate editor coordinating the review of this article and approving it for publication was Prof. Guowen Xu. *(Corresponding authors: Jiaping Gui; Cunqing Hua.)*

The authors are with the School of Computer Science, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: jgui@sjtu.edu.cn; jiruiwen@sjtu.edu.cn; mxrjkwy@sjtu.edu.cn; hongjn@sjtu.edu.cn; cqhua@sjtu.edu.cn).

Digital Object Identifier 10.1109/TIFS.2025.3587185

[1] We provide a comprehensive comparison between this work and our prior conference version [39] in Appendix 1.

data) during aggregation. The supplementary dataset is also employed to train the model, further mitigating the issue of data contamination.

In HFL-AD, we introduce a hierarchical federated learning framework, where aggregation is conducted in both upper-layer clients and the central server. Upper-layer clients possess a supplementary dataset and guide the aggregation process related to lower-layer clients. Leveraging this supplementary dataset, we obtain a post-training model that is resilient to data contamination. Compared to manually inspecting each training sample individually, compiling a supplementary dataset with a limited number (e.g., 30) of anomaly samples is significantly more cost-effective. Traditional hierarchical FL solutions mainly address issues like heterogeneous computing resources and FL acceleration [7], [8]. In contrast, our work extends this approach to provide valuable insights into anomaly detection tasks. Some previous works [6], [12] have also proposed the incorporation of a small supplementary dataset. However, these efforts have primarily focused on the centralized training scenario, which differs from the distributed FL scenario presented in this paper.

Nonetheless, we encountered three technical challenges while implementing our insights. The first challenge lies in distinguishing between normal models and contaminated models. To address this, we utilize the supplementary dataset from upper-layer clients to simulate normal and contaminated models, which provides a baseline for subsequent comparisons. The second challenge is determining the threshold between contaminated and normal models. To tackle this challenge, we adopt a learning strategy in upper-layer clients, simulating two different scenarios: unsupervised learning with the same settings as lower-layer clients, and semi-supervised learning guided by the labeled supplementary dataset. Then, we calculate the similarity score between models trained in these different scenarios, and use the average similarity score among all upper-layer clients to determine the threshold. The third challenge is resolving threshold dynamics during model training, especially when the similarity range of a normal model may undergo changes. To handle this challenge, we introduce a dynamic threshold mechanism. Unlike FLTrust [11], which employs a static threshold for aggregation, our approach calculates the threshold dynamically in each global training round, which adapts to the vast diversity of normal traffic behaviors under varying network conditions. To accomplish this, we utilize models trained on various batches of data, encompassing both normal data and contaminated (supplementary) data, for comparison. This ensures contaminated models can be promptly identified and filtered out.

We evaluated HFL-AD through extensive experiments. The results demonstrate that a supplementary dataset containing labeled anomaly samples is more beneficial than a clean dataset in identifying contaminated model updates. Across a range of contamination ratios from 0% to 8%, our approach achieved higher AUC and F1-score compared to baseline approaches. Specifically, in our experiments conducted on three different network traffic datasets, HFL-AD attained an average F1-score of 0.994 when 1/3 of the participants possessed a dataset with a contamination ratio of 4%, whereas the average F1-scores of all baseline approaches fell below 0.975.

Our contributions can be summarized as follows:

- We proposed HFL-AD, a novel approach to training an anomaly detection model on network traffic using federated learning. This approach addresses the issues of limited data diversity, data contamination, and the leakage of sensitive data.
- We gained insights that, to identify contaminated model updates in federated anomaly detection, a supplementary dataset outperforms a clean dataset.
- We demonstrated the effectiveness of our approach through numerous experiments. The results indicate that our approach maintains high accuracy, F1-score, and AUC value across a contamination ratio ranging from 0% to 8%.

**Open Science.** We have also released the source code and experimental configurations at https://github.com/realllriven/HFL-AD to facilitate further research.

## II. Related Work

### A. Hierarchical Federated Learning

With aggregation conducted in more than one layer, hierarchical FL has garnered increasing attention in recent years. Wang et al. [7] propose FedCH to address client heterogeneity in FL. Lim et al. [8] introduce a hierarchical game framework for dynamic resource allocation in self-organizing FL networks. Xu et al. [32] design CAHFL to improve hierarchical FL accuracy under unbalanced data distribution. Zhang et al. [33] focus on solving the problem of device scheduling and assignment in hierarchical FL. Deng et al. [43] proposed a hierarchical FL framework that optimizes edge-level data distribution, significantly reducing communication costs while maintaining model accuracy. Unlike these works, we propose leveraging hierarchical FL to address data contaminated problems in anomaly detection.

### B. Federated Learning-Based DDoS Detection

To preserve client privacy, researchers have leveraged FL to detect DDoS attacks. Mateus et al. [27] propose a DDoS detection system based on FL within SDN networks. They trained various classification models to categorize DDoS attack traffic and discovered that LSTM performed optimally. Doriguzzi-Corin et al. [28] devise an FL-based adaptive DDoS detection mechanism. It addresses convergence issues in dynamic cybersecurity scenarios. Fotse et al. [46] proposed an efficient FL method to combat DDoS attacks in large-scale SDN-based networks, especially in multi-controller architectures. Xu et al. [26] utilize an FL framework tailored for DDoS attack detection in blockchain networks, enabling clients to train the model without excessive resource consumption. Pourahmadi et al. [4] introduce an auto-encoder into FL to improve model performance through an outlier exposure technique. While all these efforts employ FL for DDoS attack detection, they overlook the issue of data contamination that HFL-AD aims to address.

### C. Data Contamination in Anomaly Detection

Data contamination refers to the scenario where traffic data is normally collected but may contain anomalous samples. Data contamination can lead to a significant decline in model performance [6]. To address data contamination, Qiu et al. [15] first estimate the contamination ratio and then jointly infer the binary label of each training sample during model parameter updates. Wu et al. [6] introduce an extra dataset containing a few anomaly samples. Zhang et al. [31] utilize the extra dataset as a validation set to label contaminated samples. Kim et al. [29] design an iterative learning framework with dynamically decreasing weights for training samples. Su et al. [30] introduce an extra dataset to train a bidirectional GAN. This GAN includes a discriminator capable of distinguishing between normal, anomalous, and generated samples. All these works tackle the data contamination issue in a centralized manner; however, they lack the capability to protect data privacy. In contrast, HFL-AD leverages FL to preserve the privacy of both collected data and supplementary data.

The most similar work to ours is that of Sanon et al. [47]. The authors showed that median aggregation can mitigate the influence of malicious data sources in FL by reducing the impact of abnormal updates. However, their approach does not identify or eliminate malicious clients, allowing polluted data to persist in the global model. As a result, the model exhibits significant robustness and reliability deficiencies when dealing with large-scale or stealthy DDoS pollution.

Overall, in the context of DDoS detection, limited data diversity, data contamination, and sensitive data leakage are three key challenges that existing approaches struggle to address effectively. This motivates us to propose HFL-AD, a novel hierarchical federated learning-based solution. By leveraging a small supplementary dataset, HFL-AD effectively detects DDoS traffic while filtering out contaminated model updates. In Section III, we further elaborate more on the background and motivation.

## III. Background and Motivation

In this section, we formally describe anomaly detection for network traffic, focusing on data contamination, to justify federated learning.

### A. Traditional Anomaly Detection on Network Traffic

The procedure of a traditional anomaly detection task on network traffic is as follows. Given a dataset $D_N$ comprised of normal samples, the goal of unsupervised anomaly detection is to train a model $\mathcal{M}$ to learn the patterns within $D_N$. This model $\mathcal{M}$ assigns an anomaly score $L(x;\theta)$ to each input sample $x$, where $\theta$ represents the learnable parameters of $\mathcal{M}$.

The training of $\mathcal{M}$ requires minimizing the anomaly scores of the samples in $D_N$. The loss function is given by:

$$\frac{1}{n}\sum_{i=1}^{n} L(x_i;\theta), \tag{1}$$

where $n$ represents the number of samples in $D_N$. The gradient descent algorithm is employed to iteratively update the model parameters $\theta$ as follows:

$$\theta(k+1) = \theta(k) - \eta \cdot \frac{1}{n}\sum_{i=1}^{n} \nabla_\theta L(x_i;\theta(k)), \tag{2}$$

where $k$ represents the training round and $\eta$ represents the learning rate. $g(k+1) = \theta(k+1) - \theta(k)$ is referred to as the model gradient for this iteration.

Model $\mathcal{M}$'s performance depends on $D_N$'s quality. Ideally, $D_N$ contains only normal samples, so $\mathcal{M}$ learns to assign them low anomaly scores. However, if $D_N$ inadvertently contains a few anomaly samples, the performance of $\mathcal{M}$ will be compromised. Manual inspection of $D_N$ is impractical, undermining unsupervised anomaly detection's purpose.

To address the aforementioned data contamination problems, some prior works [6], [12] proposed leveraging a small supplementary dataset, denoted as $D_A$, which comprises a limited number of anomaly samples. In the context of intrusion detection, security experts can assist in preparing this supplementary dataset by labeling a small number of attack flows.

By maximizing the anomaly scores of samples in $D_A$, the issue of data contamination can be alleviated. Formally, let $L_a(x;\theta)$ be a monotonically decreasing function of the anomaly score $L(x;\theta)$, such as $L_a(x;\theta) = -L(x;\theta)$ or $L_a(x;\theta) = \frac{1}{L(x;\theta)}$. The new loss function is given by:

$$\frac{1}{n+n_a}\sum_{i=1}^{n+n_a}((1-y_i)\cdot L(x_i;\theta) + y_i \cdot L_a(x_i;\theta)), \tag{3}$$

where $n_a$ represents the number of anomaly samples in $D_A$. The binary value $y_i$ is used to indicate the origin of $x_i$: $y_i = 0$ denotes that $x_i$ comes from the (potentially contaminated) dataset $D_N$, and $y_i = 1$ denotes that $x_i$ comes from the dataset $D_A$.

The aforementioned method can effectively mitigate the issue of data contamination on a single device. However, in real-world scenarios, training data often originates from multiple devices. Maintaining a small dataset with anomaly samples on each device, if not impossible, poses significant practical challenges. Moreover, traditional centralized training uses this method but cannot protect data privacy. This motivates us to propose a novel approach that can handle data diversity, privacy protection, and data contamination.

### B. Federated Learning (FL) for Anomaly Detection

FL can be utilized to train a global model for anomaly detection without sharing clients' private data. Suppose there are in toal $c$ clients. To train a global detection model among these clients without disclosing their sensitive data, the following three steps are iteratively executed:

1) The central server disseminates the global model parameters $\theta_{k-1}$ (where $\theta_0$ represents the randomly initialized model parameters) to all clients.
2) Each client updates its local model parameters using $\theta_{k-1}$ and its respective loss function $Loss_i$ derived from its

TABLE I
COMPARISON BETWEEN PREVIOUS WORKS AND OUR WORK

| Solutions | Research goal | | | Methods | | | Datasets for evaluation | |
|---|---|---|---|---|---|---|---|---|
| | Solving data contamination | Solving lack of data diversity | Preserving data privacy | Supplementary dataset | Federated learning | Hierarchical structure | Network traffic | Other types of datasets |
| [12] [6] [30] | ✓ | × | × | ✓ | × | × | × | ✓ |
| [15] | ✓ | × | × | × | × | × | ✓ | ✓ |
| [36] | ✓ | × | × | ✓ | × | × | ✓ | ✓ |
| [7] [32] [33] | × | ✓ | ✓ | × | ✓ | ✓ | × | ✓ |
| [5] [35] | × | × | × | × | × | × | ✓ | × |
| [3] [22] [27] [28] | × | ✓ | ✓ | × | ✓ | × | ✓ | × |
| [4] | × | ✓ | ✓ | ✓ | ✓ | × | ✓ | × |
| **HFL-AD (Ours)** | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |

local training dataset, obtaining $\theta_{k,i}, i = 1, 2, \ldots, c$, which are then uploaded to the central server.

3) On the server side, all local model parameters $\theta_{k,i}, i = 1, 2, \ldots, c$ are aggregated according to a predefined aggregation strategy. The result of this aggregation constitutes the new global model parameters $\theta_k$, which are then disseminated in the subsequent iteration.

By repeating the above steps iteratively, the global model gradually converges. Upon completion of the model training, the final aggregated global model is leveraged for anomaly detection on the test dataset (i.e., new network traffic data).

In the aforementioned FL scenario, it is possible that the network traffic collected (pertaining to $D_N$) on each client may be contaminated and contain malicious traffic, ultimately resulting a substantial decline in model performance [6]. Existing FL solutions primarily concentrate on tasks such as classification (e.g., Byzantine-robust FL) [10], [16] and word prediction [11], but cannot handle anomaly detection. In Section VII, we demonstrate that, despite their effectiveness in defending against malicious clients, traditional robust aggregation algorithms (such as Krum [9], Trimmed mean [10], FLTrust [11], and RFA [34]) fail to address data contamination in anomaly detection tasks. This is attributed to the fact that excluding model updates trained on contaminated datasets differs from excluding model updates deliberately crafted by malicious clients. Firstly, malicious clients intentionally craft model updates to disrupt the training process, whereas data contamination arises when attacks compromise a small fraction of collected data. Secondly, malicious clients typically strive to craft harmful model updates in every round, aiming to maximize their impact. However, in our work, each client trains their local model using a batch of data randomly sampled from the training dataset in every round. The number of contaminated samples within this batch is random.

We present a detailed comparison of our work with the existing literature in Tabel I. Therefore, a new FL aggregation strategy that minimizes the influence of contaminated model updates is desired for the anomaly detection of DDoS traffic. To achieve this, we proposed a hierachical FL approach, which we describe in Section VI.

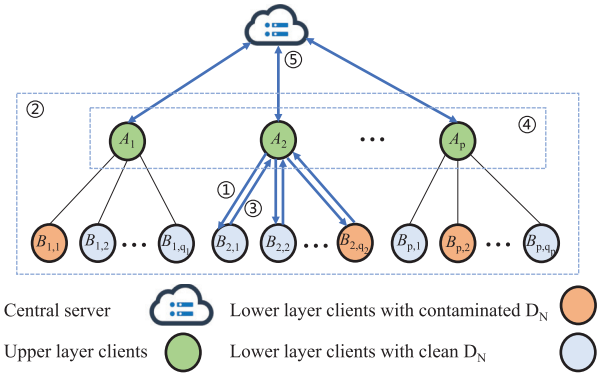



Fig. 1. Hierarchical structure in HFL-AD. Every round of federated learning consists of five steps: (1) Lower-layer clients obtain the current global model parameters. (2) All clients train their models using local data. (3) Each lower-layer client sends its training results to its corresponding upper-layer client within the same group. (4) Each upper-layer client exchanges model information with other upper-layer clients and aggregates all the training results from its group accordingly. (5) Each upper-layer client transmits the aggregated results to the central server. Finally, the central server aggregates these results and distributes the updated global model back to the upper-layer clients.

## IV. THREAT MODEL AND ASSUMPTIONS

We consider an FL system comprising a central aggregation server and a collection of distributed clients that belong to two distinct layers, as depicted in Fig. 1. We assume there are no malicious clients that intend to disrupt the training process on purpose by crafting model updates. Among all clients, the local traffic data may be contaminated, i.e., it may contain a small proportion of attack traffic. Only clients in the upper layer (as shown in the figure) can obtain annotated attack traffic from security analysts. For communications between clients or between clients and the central server, only model information is transmitted, thus protecting privacy.

## V. APPROACH OVERVIEW

We realize our strategy as a framework, HFL-AD, of which the workflow is shown in Fig. 1. It comprises three layers: lower-layer clients, upper-layer clients, and a central server. Each upper-layer client coordinates a few (e.g., 6) lower-layer clients. Regarding the training data, each lower-layer

client has only local normal traffic dataset ($D_N$) that may be potentially contaminated, whereas each upper-layer client has a small supplementary dataset ($D_A$) containing labeled anomaly samples, in addition to normal traffic data ($D_N$). The key insight is that benign traffic far outweighs attack traffic in the real world. By adopting a hierarchical structure, HFL-AD is viable for extension to a large network. Below, we outline the working procedure of HFL-AD.

At the beginning of each global round, the central server disseminates the global model to the upper-layer clients, each of which then distributes the global model to its lower-layer clients within the same group (step ① in Fig. 1). Lower-layer clients train the model unsupervised on local data, while upper-layer clients employ both local data and the labeled supplementary dataset for semi-supervised training (step ②). Once local training is completed, each lower-layer client sends the trained model to its corresponding upper-layer client within the same group (step ③). Each upper-layer client then trains additional models, calculates cosine similarities among them, and exchanges these values with peers to derive a contamination-filtering threshold. The details of this step (step ④) will be elaborated in Section VI-B. Finally, each upper-layer client calculates the average of all retained models and uploads the average model to the central server for global aggregation (step ⑤), completing one global training round.

## VI. Design

In this section, we delve into the design details of HFL-AD, including its hierarchical structure and the aggregation algorithm.

### A. Hierarchical Structure

FL training data in real-world scenarios may contain attack traffic, though most data is benign. As outlined in Section II-C, data contamination can significantly impact model performance. Checking all data samples individually to eliminate attack traffic is infeasible due to the immense laborforce required. A more viable solution is to filter out contaminated model updates trained on the collected data, ensuring only uncontaminated model updates are aggregated on the server side. However, distinguishing between clean and contaminated model updates is challenging, as the number of labeled attack samples is limited. A client with uncontaminated $D_N$ cannot reliably identify clean model updates without additional information (see Section VII-B).

To address the aforementioned challenge, we propose leveraging a hierarchical structure that divides all clients into two layers: the **lower layer** and the **upper layer**. In the lower layer, clients that lack the supplementary dataset $D_A$ train their models in each round solely using $D_N$. In contrast, clients in the upper layer possess $D_A$, and employ both $D_N$ and $D_A$ for model training.

As depicted in Fig. 1, suppose there are a total of $p$ clients in the upper layer, namely $A_1, A_2, \ldots, A_p$. Each $A_i$ manages a few lower-layer clients $B_{i,1}, B_{i,2}, \ldots, B_{i,q_i}$ for $i = 1, 2, \ldots, p$. We refer to $A_i$ and $B_{i,1}, B_{i,2}, \ldots, B_{i,q_i}$ as belonging to the same group, for $i = 1, 2, \ldots, p$. In the $k$-th round of the federated learning process, the following steps are performed:

1) Each lower-layer client $B_{i,j}$ receives the current model parameters $\theta(k-1)$ from its upper-layer client $A_i$.
2) Each lower-layer client $B_{i,j}$ trains a local model with parameters $\theta_{i,j}(k)$ using its dataset $D_N$. Meanwhile, each upper-layer client $A_i$ trains a local model with parameters $\theta_{i,0}(k)$ using both its datasets $D_N$ and $D_A$. The loss function for lower-layer clients is given in Eq. 1, whereas the loss function for upper-layer clients is provided in Eq. 3.
3) Each lower-layer client $B_{i,j}$ sends the local model parameters $\theta_{i,j}(k)$ to its upper-layer client $A_i$.
4) Each upper-layer client $A_i$ aggregates $\{\theta_{i,0}(k), \theta_{i,1}(k), \ldots, \theta_{i,q_i}(k)\}$ and obtains $\theta_i(k)$. This step will be referred to as *lower-layer aggregation* in the rest of the paper.
5) A central server aggregates $\{\theta_1(k), \theta_2(k), \ldots, \theta_p(k)\}$ sent by the upper-layer clients, obtains the result $\theta(k)$. This step will be referred to as *upper-layer aggregation* in the rest of the paper.

In the procedure outlined above, the lower-layer aggregation plays a crucial role. It excludes model updates trained on contaminated datasets, utilizing a mechanism that we will outline in Section VI-B.

### B. Aggregation Algorithm

The goal of the proposed aggregation algorithm is to determine a threshold that distinguishes contaminated model updates from uncontaminated ones. This operation is conducted on upper-layer clients, where both local model updates and model updates from lower-layer clients are collected. In the high level, we aim to identify a boundary between contaminated and uncontaminated model updates by utilizing the supplementary dataset.

Intuitively, we have two choices for this additional small labeled dataset: clean traffic and attack (or contaminated) traffic, both enabling contamination detection. FLTrust [11] leverages a small, clean dataset to determine the trust score of a client model, thereby assisting in filtering out malicious clients. However, The utilization of clean traffic presents a challenge in addressing the data contamination issue. Normal traffic behavior exhibits greater diversity compared to malicious traffic, given that the majority of traffic collected in real-world scenarios pertains to normal network behaviors. Models trained on a different type of normal traffic data might be misidentified as contaminated and consequently excluded from the upper-layer aggregation. The absence of these models ultimately impacts the global model's performance in anomaly detection (see Section VII-B). Conversely, the behavioral patterns of attack traffic deviate more significantly from those of clean (i.e., uncontaminated) traffic than from contaminated traffic. This enables models trained on clean normal traffic to be included in the upper-layer aggregation, enhancing the performance of the global model in DDoS detection.

Therefore, we leverage labeled attack traffic as supplementary data. To make such a comparison, we leverage cosine similarity [11], a widely adopted approach to measure vector differences. In this process, we encounter a challenge: as the training progresses, the similarity range of a normal model will change. Existing solutions, such as FLTrust, which focus

on detecting malicious clients, fail to account for this circumstance. These solutions determine whether to incorporate a new model into aggregation based on a static threshold. For example, FLTrust only aggregates models with a similarity score above zero.

To address the aforementioned challenge, we dynamically adjust the threshold each global training round by comparing models trained on mixed normal and contaminated (supplementary) data batches. The threshold is dynamically calculated in each round, ensuring that contaminated models can be identified and filtered out in a timely manner. Compared to static thresholds, this dynamic threshold mechanism adapts to the vast diversity of normal traffic behaviors, demonstrating that HFL-AD can be applied to varying network conditions.

In the high level, the proposed aggregation algorithm aims to filter out contaminated model updates (e.g., from clients with poisoned data) in federated learning (FL) by dynamically adjusting a similarity threshold. Just as a thermostat adjusts room temperature based on real-time feedback, our threshold adapts to model behavior in each round. Unlike static thresholds (e.g., FLTrust), our method adapts to changes in model behavior during training, ensuring robustness against data contamination (e.g., DDoS attack traffic).

The process involves two main steps, executed by upper-layer clients:

*1) Dynamic Threshold Calculation:* In this step, HFL-AD determines a threshold to separate contaminated vs. normal updates. To do it, HFL-AD computes a threshold $T(k)$ in each global round $k$ to identify suspicious updates as follows: i) **Simulating Contaminated Updates.** Each upper-layer client $A_i$ first trains a "contaminated" model $\theta_i^{t1}(k)$ using a mix of normal data $D_N$ and attack data $D_A$. This mimics potential malicious behavior, creating a baseline for comparison. We leverage attack traffic as reference because normal traffic is diverse and attack traffic provides a sharper signal for contamination. Attack traffic prevents false exclusion of benign clients with atypical normal data. ii) **Computing Cosine Similarity.** Each upper-layer client then compares the contaminated model's gradient $g_i^{t1}(k)$ with the local (normal) model's gradient $g_{i,0}(k)$ using the cosine similarity. This similarity acts as a "Trust Meter" to measures how much a client's update aligns with the expected direction (like a compass pointing to "normal"). Low similarity suggests deviation from expected behavior. iii) **Collaborative Thresholding.** Clients share similarity scores $s_i^{t1}(k)$ and compute the mean $\mu(k)$ and standard deviation $\sigma(k)$. The threshold is set to $T(k) = \mu(k) + m \cdot \sigma(k)$, where $m$ is a hyperparameter. We introduce this dynamic threshold because normal behavior varies across rounds, to which static thresholds (e.g., FLTrust's fixed $T = 0$) fail to adapt.

Algorithm 1 illustrates the process of dynamic threshold calculation. Specifically, each client $A_i$ in the upper layer only trains one model, $\theta_i^{t1}(k)$, which utilizes the loss function in Eq. 1 to mimic a contaminated model update (Line 2). After that, the cosine similarity $s_i^{t1}(k)$ is calculated and exchanged (Lines 3-4), where

$$S(a, b) = \frac{a^T \cdot b}{|a| \cdot |b|}. \tag{4}$$

**Algorithm 1** The Upper-Layer Client $A_i$ Dynamically Determines the Threshold in Round $k$

**Input**: $D_N$ and $D_A$ on the upper-layer client $A_i$, the aggregated model parameters $\theta(k-1)$ from the previous round, the local training results $\theta_{i,0}(k)$ of the current round, and the hyper-parameter $m$.

**Output**: Threshold $T(k)$ in round $k$.

1 $g_{i,0}(k) = \theta_{i,0}(k) - \theta(k-1)$

// Sample a batch $X_1$ from $D_N$ and $D_A$

2 $\theta_i^{t1}(k) = \theta(k-1) - \eta \cdot \frac{1}{|X_1|} \sum_{i=1}^{|X_1|} \nabla_\theta L(x_i; \theta(k-1))$, where the loss function is in Eq. 1

3 $g_i^{t1}(k) = \theta_i^{t1}(k) - \theta(k-1)$

4 $s_i^{t1}(k) = S(g_i^{t1}(k), g_{i,0}(k))$

5 Initialize an empty list $P$

6 Append $s_i^{t1}(k)$ to $P$

7 **for** $j \in \{1, 2, \ldots, p\} \setminus \{i\}$ **do**

8 Send $s_i^{t1}(k)$ to the upper layer client $A_j$

9 Receive $s_j^{t1}(k)$ from $A_j$

10 Append $s_j^{t1}(k)$ to $P$

11 **end**

12 $\mu(k) = MEAN(P)$

13 $\sigma(k) = STD(P)$

14 $T(k) = \mu(k) + m \cdot \sigma(k)$

With a list of the estimated cosine similarities of contaminated gradients, the mean $\mu(k)$ and the standard deviation $\sigma(k)$ are calculated (Lines 5-13). Then, $\mu(k) + m\sigma(k)$ is used as $T(k)$, where $m$ is a hyper-parameter.

*2) Model Aggregation:* HFL-AD filters and aggregates updates based on the threshold. In this step, only trustworthy updates (with similarity $> T(k)$) are aggregated. This process consists of two substeps: i) **Collecting Updates.** The upper-layer client $A_i$ first gathers models from lower-layer clients $B_{i,j}$ along with its own local model $\theta_{i,0}(k)$. ii) **Filtering and Aggregating.** For each model, HFL-AD computes its similarity to $\theta_{i,0}(k)$. HFL-AD then retains only those updates where $s_{i,j}(k) > T(k)$ and aggregates them using FedAvg. The key difference from FLTrust is that our method includes $\theta_{i,0}(k)$ in aggregation (as it is trained to maximize attack detection).

We outline the entire procedure that an upper-layer client $A_i$ follows during the global training round $k$ in Algorithm 2. Initially, $A_i$ receives the global model parameters $\theta(k-1)$ from the central server (Line 1) and disseminates them to its lower-layer clients (Lines 2-4). Subsequently, $A_i$ trains a model locally (Line 5) and receives the newly trained models from its lower-layer clients upon their completion of training (Lines 6-8). Then, $A_i$ computes the threshold $T(k)$ through Algorithm 1 (Line 9). Following this step, $A_i$ calculates the similarity score between its locally trained model $\theta_{i,0}(k)$ and all collected models (Lines 10-13), and aggregates all models whose cosine similarity exceeds $T(k)$ using FedAvg (Lines 14-20). Notice that unlike the existing work FLTrust, where the model trained on the supplementary dataset does not participate in the model aggregation, HFL-AD utilizes $\theta_{i,0}(k)$ as a basis for calculating the similarity score and includes it in the model aggregation. This is because the training of $\theta_{i,0}(k)$

**Algorithm 2** The Entire Procedure Followed by the Upper-Layer Cilent $A_i$ in Round $k$

**Input**: $D_N$ and $D_A$ on the upper-layer client $A_i$, the aggregated model parameters $\theta(k-1)$ from the previous round (i.e., $k-1$).
**Output**: The aggregated model parameters $\theta(k)$ in round $k$.

Receive $\theta(k-1)$ from the central server
**for** $j = 1, 2, \ldots, q_i$ **do**
  Send $\theta(k-1)$ to $B_{i,j}$
**end**
// Sample a batch $X_0$ from $D_N$ and $D_A$
$\theta_{i,0}(k) = \theta(k-1) - \eta \cdot \frac{1}{|X_0|} \sum_{i=1}^{|X_0|} \nabla_\theta L(x_i; \theta(k-1))$, where the loss function is in Eq. 3
**for** $j = 1, 2, \ldots, q_i$ **do**
  Receive $\theta_{i,j}(k)$ from $B_{i,j}$
**end**
Get $T(k)$ using Algorithm 1
**for** $j = 0, 1, 2, \ldots, q_i$ **do**
  $g_{i,j}(k) = \theta_{i,j}(k) - \theta(k-1)$
  $s_{i,j}(k) = S(g_{i,j}(k), g_{i,0}(k))$
**end**
$U = \emptyset$
**for** $j = 0, 1, 2, \ldots, q_i$ **do**
  **if** $s_{i,j}(k) > T(k)$ **then**
    $U = U \cup \{\theta_{i,j}(k)\}$
  **end**
**end**
$\theta_i(k) = \frac{1}{|U|} \sum_{\theta_j \in U} \theta_j$
Send $\theta_i(k)$ to the central server

aims to maximize the abnormal score of labeled attack traffic in $D_A$, which is beneficial for resisting data contamination. Finally, $A_i$ sends the resulting parameters $\theta_i(k)$ to the central server, which subsequently conducts the model aggregation (i.e., FedAvg).

### C. Theoretical Analysis

In this section, we investigate the impact of HFL-AD's hierarchical aggregation on its model robustness and convergence.

*1) Robustness:* The hierarchical aggregation in HFL-AD improves robustness by localized contamination filtering and dynamic threshold adaptivity. Specifically, upper-layer clients exclude contaminated updates before global aggregation, reducing the propagation of biased gradients, whereas the threshold $T(k)$ ensures tighter bounds on gradient deviations compared to static thresholds (e.g., FLTrust). **Proof sketch.** A model update $\theta_{i,j}(k)$ from lower-layer client $B_{i,j}$ is retained if its cosine similarity with the upper-layer client's gradient $g_{i,0}(k)$ satisfies: $S_{i,j}(k) = \frac{g_{i,j}(k)^T g_{i,0}(k)}{\|g_{i,j}(k)\| \|g_{i,0}(k)\|} > T(k)$. Let $g_{clean}$ be the gradient of an uncontaminated model and $g_{cont}$ be that of a contaminated model. Due to the divergence in loss landscapes, $S(g_{cont}, g_{i,0})$ tends to be smaller than $S(g_{clean}, g_{i,0})$. By setting $T(k) = \mu(k) + m \cdot \sigma(k)$, we ensure: $\mathbb{P}(S(g_{cont}, g_{i,0}) \leq T(k)) \geq 1 - \Phi(m)$, where $\Phi(m)$ is the CDF of the standard normal distribution. For $m = 2$, this probability exceeds 95%.

*2) Convergence Guarantees:* The convergence of traditional hierarchical federated learning (HFL) has been extensively analyzed and theoretically guaranteed [7], [40], [41], [42]. Traditional HFL corresponds to HFL-AD without any contaminated models being filtered out. In HFL-AD, the lower-layer aggregation mechanism ensures that excluded updates are high-probability outliers, thereby preserving the unbiasedness of aggregated gradients. Furthermore, the upper-layer aggregation in HFL-AD maintains this unbiased property through its use of FedAvg. Therefore, HFL-AD is guaranteed to converge to a stationary point of the global loss function.

## VII. Evaluation

We implemented a prototype of HFL-AD and evaluated it using three representative datasets of network traffic in the DDoS detection task. These datasets are: CICDDoS2019 [17], CICIoT2023 [18], and CICIoMT2024 [19]. In this section, we first describe our experimental setup, including hyper-parameters and settings, training models in the lower-layer aggregation, and the hardware configurations utilized for evaluation.

To evaluate HFL-AD, we aim to answer the following research questions:

**RQ 1**: Are supplementary datasets and dynamic thresholds necessary to distinguish between clean and contaminated model updates?
**RQ 2**: How effective is HFL-AD in detecting DDoS attacks when the training data is contaminated?
**RQ 3**: Is HFL-AD scalable to accommodate a larger number of clients within the FL framework, anomaly detection of non-traffic data, and alternative upper-layer aggregation algorithms?

### A. Experimental Setup

In both upper-layer and lower-layer clients, we adopt Neural Transformation Learning (NTL) [14] as the local anomaly detection model. NTL comprises $K$ learnable transformations, denoted as $Tr_i(\cdot)$, along with a shared encoder, $f_\phi(\cdot)$. The anomaly score for an input $x$ is calculated as follows:

$$L(x; \theta) = -\sum_{i=1}^{K} \log d_i(x), \tag{5}$$

where

$$d_i(x) = \frac{h(Tr_i(x), x)}{h(Tr_i(x), x) + \sum_{l \neq i} h(Tr_i(x), Tr_l(x))}, \tag{6}$$

$$h(a, b) = \exp(S(f_\phi(a), f_\phi(b))/\tau), \tag{7}$$

and $\tau$ is the temperature parameter, $\theta$ denotes all the learnable parameters, encompassing the $K$ transformations and one encoder. In our experiments, we set $\tau = 0.1$ in NTL, and chose Adam as the optimizer with a learning rate of 0.01. The system comprises three upper-layer clients, each managing six

lower-layer clients. Among these, seven clients (one upper-layer and two lower-layer per group) were intentionally fed contaminated training data.

To train the model utilizing the supplementary dataset $D_A$, we refer to the related work [15] by employing the following loss function:

$$L_a(x;\theta) = -\sum_{i=1}^{K} \log\left(1 - d_i(x)\right), \tag{8}$$

where $d_i(x)$ retains the same meaning as in Eq. 5. By substituting Eq. 5 and Eq. 8 into Eq. 1 and Eq. 3, we obtain the loss function utilized by the lower-layer clients and upper-layer clients, respectively.

We leverage FedAvg for aggregating models in the upper-layer aggregation. For the lower-layer aggregation, apart from the algorithm proposed in HFL-AD (Section VI-B), we further assess FedAvg [13], Krum [9], Trimmed mean [10], FLTrust [11], and RFA [34] as baseline methods for comparison. Notice that the latter four algorithms are specifically designed for robust aggregation. We adapt them to address the data contamination issue in our work. Below, we briefly introduce these four baseline methods, assuming that each upper-layer client collects in total $n$ trained models for aggregation.

- **Krum:** Among $n$ models, Krum selects one model that is "nearest to the rest $n-z-2$ models" as the aggregation result, assuming that at most $z$ models are malicious or contaminated.
- **Trimmed mean:** It utilizes a coordinate-wise aggregation strategy, assuming that at most $z$ models are contaminated. For each parameter, Trimmed mean removes the $z$ largest and $z$ smallest value, and uses the average of the remaining $n-2z$ values as the aggregation result for that parameter.
- **FLTrust:** This algorithm collects a clean root dataset at the server. A gradient trained on this root dataset is used as a basis. The aggregation result is the weighted average of $n$ normalized gradients, where the weights are determined by the cosine similarity between the basis and each of the $n$ gradients (excluding gradients with a cosine similarity less than 0).
- **RFA:** RFA calculates the geometric median of $n$ models as the aggregation result. To calculate the geometric median, RFA employs an iterative method.

Compared to the baselines above, HFL-AD introduces novelty in the following aspects: dynamic thresholding for contamination detection, leveraging attack traffic (rather than clean data) for robustness, integration of contamination-aware training, and hierarchical robustness. Table IV provides a comparison based on these aspects.

Since there are at most three models contaminated in each client group during our evaluation, we set $z = 3$ for both Krum and Trimmed mean. As for FLTrust, each upper-layer client possesses a dataset $D_F$ comprising 30 labeled normal samples (i.e., traffic flows), which serves as the root dataset. We chose this size to be consistent with the labeled supplementary dataset on each upper-layer client. Table II details all dataset-independent settings. In addition, we list the dataset-dependent settings in Table III. These parameters were empirically determined to ensure optimal model performance.

TABLE II
DATASET-INDEPENDENT SETTINGS USED IN EXPERIMENTS

| Description | Setting |
|---|---|
| $\tau$ in NTL | 0.1 |
| $m$ in Algorithm 1 | 2 |
| Learning rate | 0.01 |
| Optimizer | Adam |
| # upper-layer clients | 3 |
| The upper-layer clients with contaminated $D_N$ | $A_1$ |
| $z$ in Krum and Trimmed mean | 3 |
| Size of $D_A$ or $D_F$ on each upper-layer client | 30 |
| # lower-layer clients in each group | 6 |
| Lower-layer clients with contaminated $D_N$ | $B_{i,1}, B_{i,2}, i = 1, 2, 3$ |

All experiments were conducted on a system equipped with an Intel(R) Core(TM) i9-14900K CPU and an NVIDIA GeForce RTX 4090 GPU, using the PyTorch framework and Python 3.9.

### B. RQ1: Necessity of Supplementary Datasets and Dynamic Thresholds

To answer this research question, we first present empirical evidence that merely utilizing a clean $D_N$ is insufficient to distinguish between clean and contaminated model updates. Then, we underscore the necessity of dynamically determining the threshold. Across all datasets, we observe consistent results, and for the purpose of illustration in this subsection, we utilize findings from CICDDoS2019.

To do the analysis, we assess the gradients of trained anomaly detection models in three distinct scenarios: (1) using only a clean $D_N$, (2) using both $D_A$ and clean $D_N$, and (3) using only a contaminated $D_N$. For brevity, we introduce the following terminology to represent these scenarios.

- **type0:** The model is trained on a purely clean batch consisting of 1,024 normal samples, employing the loss function outlined in Eq. 1.
- **type1:** The model is trained on a batch that includes 994 normal samples, along with 10 samples of NetBIOS attack, 10 samples of Syn attack, and 10 samples of LDAP attack, utilizing the loss function specified in Eq. 3.
- **type2:** The model is trained on a contaminated batch with 1,003 normal samples, including 7 samples of SNMP attack, 7 samples of Portmap attack, and 7 samples of UDPLag attack, using the loss function detailed in Eq. 1. The contamination ratio within this batch is approximately 2%.

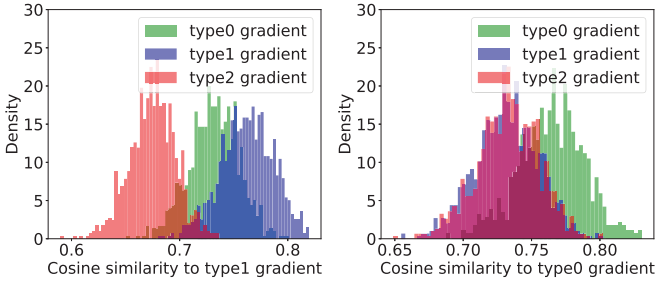
Intuitively, the type2 gradient has a negative impact on the training process. As indicated by prior work [6], the type1 gradient can alleviate the issue of data contamination to some degree. Hence, our objective is to eliminate type2 gradients and retain as many type0 and type1 gradients as possible. To understand the probability distribution of cosine similarity

TABLE III
DATASET-DEPENDENT SETTINGS USED IN EXPERIMENTS

| Description | Setting | | | |
|---|---|---|---|---|
| | CICDDoS2019 | CICIoT2023 | CICIoMT2024 | Census |
| Batch size | 1,024 | 2,048 | 1,024 | 1024 |
| # rounds per training | 800 | 1,500 | 1,000 | 500 |
| Size of $D_N$ on each client | 3,500 | 24,000 | 5,200 | 6000 |
| # input features | 66 | 46 | 45 | 508 |
| Shape of the transformation in NTL | [66, 48, 32, 66] | [46, 48, 32, 46] | [45, 32, 32, 45] | [508, 16, 16, 508] |
| Shape of the encoder in NTL | [66, 48, 32, 32, 32] | [46, 64, 32, 32, 16] | [45, 64, 32, 32, 16] | [508, 32, 16, 16] |
| # total normal samples in training set | 3500*21 | 24000*21 | 5200*21 | 6000*21 |
| # total normal samples in testing set | 40000 | 590000 | 120000 | 150000 |
| $K$ in NTL | 4 | 8 | 8 | 4 |

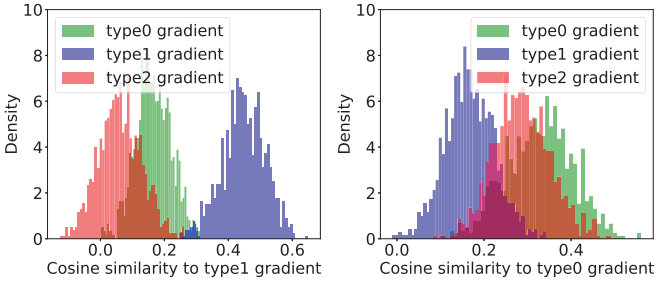
TABLE IV
COMPARISON WITH BASELINES

| **Method** | **Static/Dynamic** | **Data Used for Defense** | **Contamination Awareness** | **Hierarchical Support** |
|---|---|---|---|---|
| FLTrust | Static (fixed threshold) | Clean data ($D_F$) | No (generic malicious detection) | No |
| Krum/Trimmed Mean | Static (assumes $z$ malicious clients) | None | No (outlier removal only) | No |
| RFA | Static (geometric median) | None | No (robust to arbitrary outliers) | No |
| HFL-AD (**Ours**) | **Dynamic** (adapts per round) | **Attack data** ($D_A$) | **Yes** (explicit contamination detection) | **Yes** |




(a) Cosine similarity between type1 gradient and other gradients. (b) Cosine similarity between type0 gradient and other gradients.

Fig. 2. Differentiating capability of type0 and type1 gradients (training starts from a randomly initialized model).




(a) Cosine similarity between type1 gradient and other gradients. (b) Cosine similarity between type0 gradient and other gradients.

Fig. 3. Differentiating capability of type0 and type1 gradients (training starts from a model that has been trained on 100 clean batches).

between gradients of different types, we repeatedly sample two batches, train two gradients, and calculate their cosine similarity. This calculation is repeated 1,000 times for each "type pair."

Fig. 2 shows the histogram of cosine similarity, where the training starts from a randomly initialized model. In Fig. 2(a), we present the cosine similarity between type1 gradients and gradients of type0, type1, and type2, respectively. The choice of cosine similarity to compare gradients (type0/-type1/type2) is grounded in its widespread use for measuring directional alignment in high-dimensional spaces, particularly in federated learning (FL) robustness studies (e.g., [9], [11]). It can be observed that type1 gradients exhibit the highest cosine similarity with gradients of the same type, whereas type2 gradients demonstrate the lowest cosine similarity with type1 gradients. The overlap between type1-type1 and type1-type2 gradients is minimal.

In contrast, Fig. 2(b) depicts the cosine similarity between type0 gradients and those of type0, type1, and type2. Two observations can be drawn from this subfigure: (i) The overlap between the undesired type2 gradients (type0-type2, shown in red) and the desired type0 gradients (type0-type0, shown in green) is greater than that depicted in Fig. 2(a). (ii) The overlap between the undesired type2 gradients (type0-type2) and the desired type1 gradients (type0-type1) is substantial. Therefore, it is challenging to distinguish these two types of gradients in the type0 scenario.

We further show the histogram of cosine similarity in Fig. 3, where the training starts from a model pre-trained on 100 clean batches. From the figure, it becomes evident that the prior analysis remains valid. Therefore, we propose the following

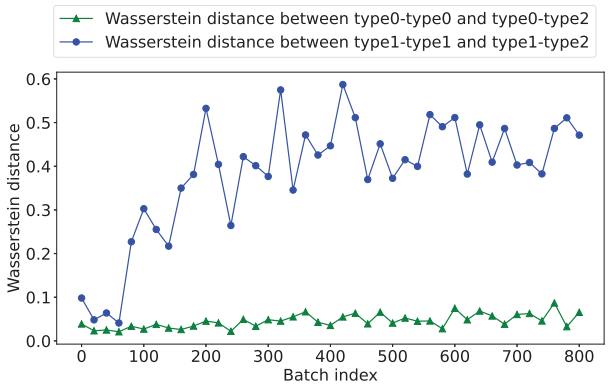



Fig. 4. Wasserstein distance of cosine similarity distributions.

aggregation rule in HFL-AD: utilizing type1 gradients as a basis, we estimate a cosine similarity threshold to exclude as many type2 gradients as possible. Specifically, we propose that clients possessing $D_A$ (pertaining to the type1 scenario) should be positioned in the upper-layer clients and tasked with excluding contaminated gradients. In addition, by comparing the corresponding cosine similarity values in Fig. 2 and Fig. 3, we observe that the appropriate threshold undergoes significant changes after training on 100 clean batches. This suggests that a dynamic threshold needs to be estimated for each training batch.

To better understand the cosine similarity distribution throughout the training process, we leverage the Wasserstein distance [38], a measurement method used to quantify the difference between two probability distributions. We chose the Wasserstein distance because it is sensitive to both shape and location shifts in probability distributions. This dual-metric approach ensures robustness against contamination by distinguishing type0/type1 and type2 updates. A larger distance value indicates that the two distributions are more distinct, facilitating a more accurate threshold determination in HFL-AD. To assess this distance, we initiate the training process using a model pre-trained on a batch range of 0 to 800. Specifically, Fig. 2 and Fig. 3 correspond to the 0th and 100th batch indices, respectively. We present the experimental results in Fig. 4, where the blue line depicts the Wasserstein distance between the cosine similarity distribution of type1-type1 gradients and the distribution of type1-type2 gradients, while the green line represents the distance between type0-type0 gradients and type0-type2 gradients. It is evident that the Wasserstein distance for the similarity distribution related to type0 gradients (green line) is significantly smaller than that of type1 gradients (blue line), aligning with our previous analysis.

Based on the above results, we conclude that even with a completely clean $D_N$ which corresponds to the type0 scenario, it is challenging to identify contaminated training updates (i.e., type2 gradients). In contrast, the type1 gradient is able to differentiate contaminated gradients from non-contaminated ones, thus making it more suitable as a basis for use in HFL-AD.

The Wasserstein distance analysis further validates our assumption that gradient distributions evolve during training (Figs. 2–3), justifying the need for adaptive thresholds. Static thresholds would fail to capture this non-stationarity.

TABLE V
STATISTICAL SIGNIFICANCE (P-VALUES) OF PERFORMANCE COMPARISONS BETWEEN HFL-AD AND BASELINE METHODS

| P-value | FedAvg | FLTrust | Krum | RFA | Trimmed Mean |
|---|---|---|---|---|---|
| Accuracy | 0.03348 | 0.03903 | 0.01682 | 0.01642 | 0.01242 |
| Precision | 0.0557 | 0.04619 | 0.04269 | 0.04899 | 0.01692 |
| Recall | 0.14148 | 0.06935 | 0.02551 | 0.1301 | 0.16332 |
| F1-score | 0.03969 | 0.04339 | 0.02037 | 0.01956 | 0.01507 |
| AUC | 0.03458 | 0.0318 | 0.0161 | 0.01218 | 0.00269 |

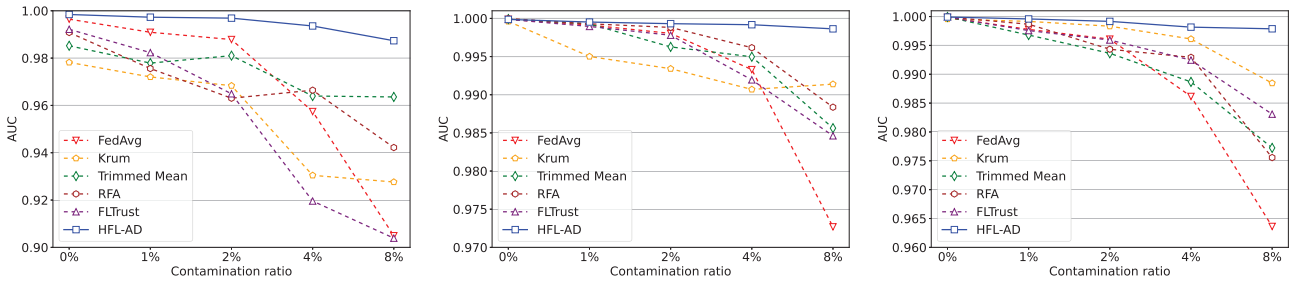
### C. RQ2: Effectiveness of HFL-AD

In this section, we demonstrate the effectiveness of our proposed method from two different perspectives. Firstly, we utilize the AUC (the area under the ROC curve) value to assess the model's performance on DDoS detection. AUC-ROC is adopted due to its threshold-agnostic nature, which is critical for unbiased evaluation under varying contamination ratios. This aligns with best practices in anomaly detection literature [6], [15]. A higher AUC value indicates better model performance, with a value of 1 representing perfect distinction between positive and negative samples. Notice that the ROC curve plots the true positive rate (TPR) against the false positive rate (FPR) for all thresholds, providing a comprehensive understanding of the model's capabilities. This also indicates the AUC value derived from the ROC curve is indepedent of threshold determination.

Fig. 5 shows the performance of various aggregation algorithms across all three datasets, with contamination ratios of 0%, 1%, 2%, 4%, and 8%. We chose these ratios because they align with empirical observations from [6] and [15], where low-to-moderate anomaly fractions (typically less than 10%) reflect realistic attack scenarios. The AUC value is evaluated for all types of DDoS attacks. From the figure, it is evident that HFL-AD achieves the highest AUC value across all contamination ratios. When the contamination ratio is 0%, indicating that all clients possess clean datasets, HFL-AD effectively leverages the model updates from all clients, yielding a result that is comparable to FedAvg. As the contamination ratio increases, HFL-AD effectively mitigates the adverse impact of contaminated model updates.

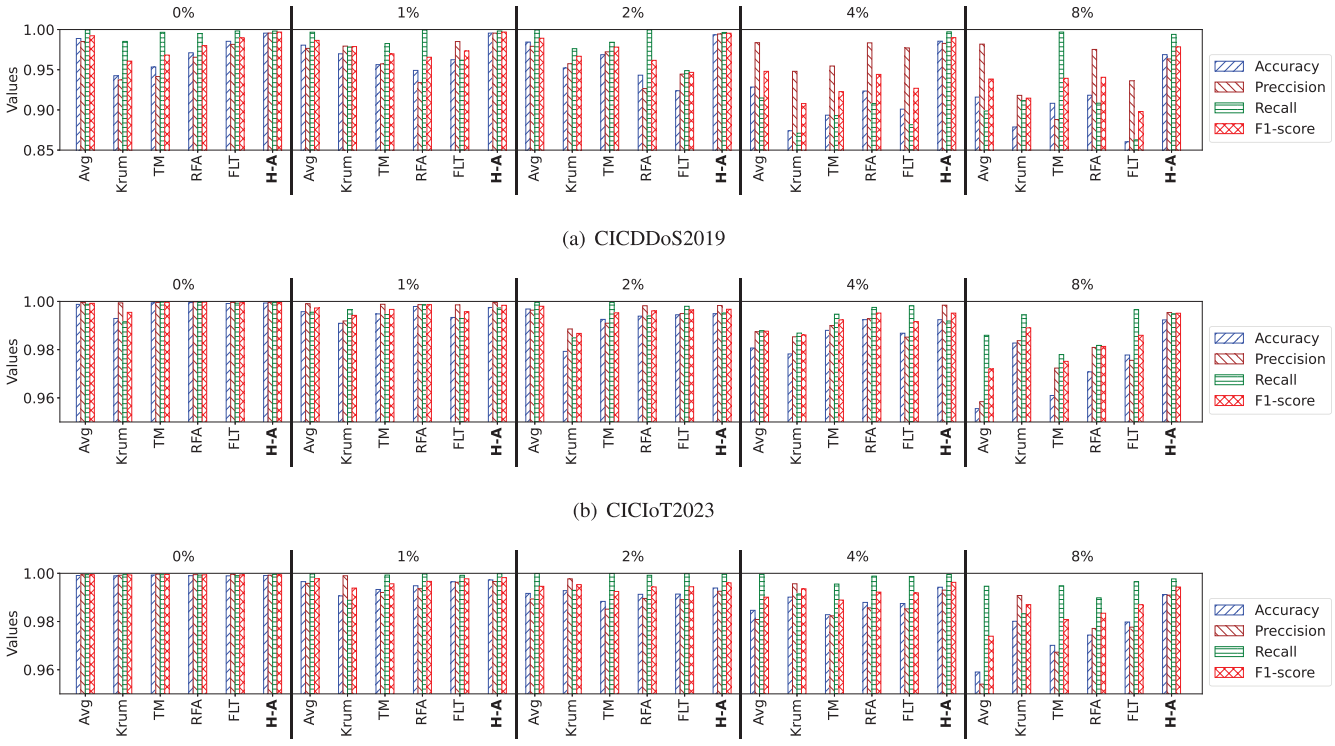
Secondly, to further evaluate models, we leverage threshold-dependent metrics, including accuracy, precision, recall, and F1-score. Specifically, we utilize a small validation dataset comprising 300 normal samples and 300 abnormal samples. We adopt the threshold that yields the highest F1-score on the validation dataset for testing. Fig. 6 presents the results of different aggregation algorithms across all contamination ratios. Overall, among all four metrics, HFL-AD achieves the best performance, particularly reflected by the F1-score, which represents a harmonious balance between precision and recall. These results underscore the effectiveness of HFL-AD.

To verify the superiority of our proposed method, we conducted two-sample t-tests comparing HFL-AD's results




(a) CICDDoS2019 (b) CICIoT2023 (c) CICIoMT2024

Fig. 5. AUC performance among different aggregation algorithms on three datasets.




(a) CICDDoS2019

(b) CICIoT2023

(c) CICIoMT2024

Fig. 6. Performance comparison among different aggregation algorithms under five contamination ratios. Notice that Avg, TM, FLT, H-A stand for FedAvg, Trimmed Mean, FLTrust, and HFL-AD, respectively.

in Fig. 5 and Fig. 6 with baseline methods. As shown in Table V, the statistical results demonstrate that HFL-AD achieves statistically significant superiority over all baselines in Accuracy, F1-score, and AUC (with *p-value* $\leq 0.05$), while showing competitive performance in Precision and Recall (where some *p-values* are slightly above 0.05).

### *D. RQ3: Scalability of* HFL-AD

We answer this research question by presenting the AUC score of HFL-AD, considering factores such as the number of lower-layer clients per group, non-traffic data for anomaly detection, and an alternative upper-layer aggregation algorithm. These factors are illustrated in Fig. 7, Fig. 8, and Fig. 9, respectively.

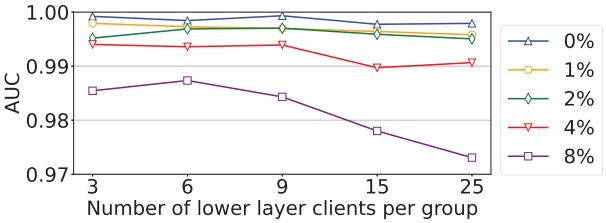



Fig. 7. AUC performance of different aggregation algorithms under varying numbers of lower-layer clients in a group.

*1) Number of Lower-Layer Clients:* The upper-layer aggregation in HFL-AD is based on FedAvg. As a result, HFL-AD can support a large number (e.g., 100+) of upper-layer clients,

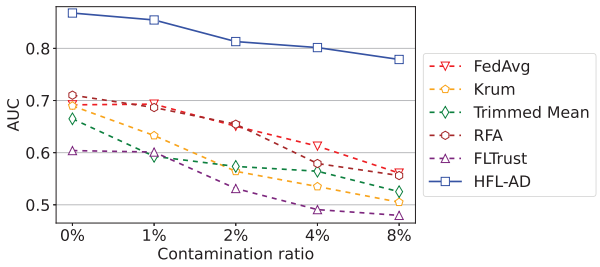



Fig. 8. AUC performance of different aggregation algorithms on the Census dataset.

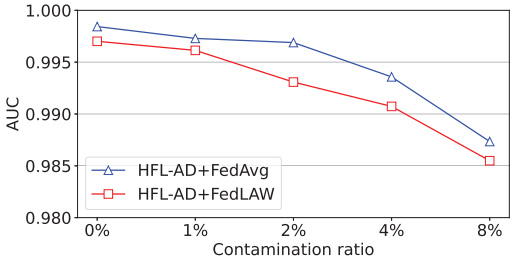



Fig. 9. AUC performance of HFL-AD under different upper-layer aggregation algorithms.

similar to other FL solutions [3], [4], [22], [27], [28]. To analyze the scalability of HFL-AD with lower-layer clients, we conduct experiments using the CICDDoS2019 dataset. We vary the number of lower-layer clients within each group to 3, 6, 9, 15, and 25, respectively. Notice that due to the dataset's limited sample size, 25 is the maximum number of lower-layer clients that can be assigned to each group to ensure viable training on each client. In this configuration, there are a total of 78 clients, with each client receiving 942 training samples. For each experiment, one-third of the clients are contaminated with malicious data for training purposes. Fig. 7 depicts the experimental results. We can see that HFL-AD maintains stable performance across various contamination ratios (ranging from 0% to 4%). When the contamination ratio reaches 8%, there is a notable decrease in HFL-AD's performance, especially when a higher number of lower-layer clients is assigned to each group. However, this performance decrement is minimal. Despite the higher contamination ratio and reduced training data per client, the AUC performance of HFL-AD still surpasses existing solutions (see Fig. 6).

*2) Applicability to Non-Traffic Data:* HFL-AD is designed to train the anomaly detection model in a federated manner. That said, beyond DDoS, HFL-AD can also detect other anomalies, such as malware traffic (see Section VIII) or non-traffic anomalies. To assess the applicability of HFL-AD to other types of datasets, we leverage the Census dataset [48], a tabular dataset pertaining to income surveys. We follow the same procedure as was employed for the traffic datasets utilized in the network intrusion detection task. The last column in Table III outlines the setting information pertinent to the Census dataset. Fig. 8 shows the performance of various aggregation algorithms on this dataset. We can see that HFL-AD remains effective when applied to non-traffic data. In contrast, existing solutions exhibit considerable performance degradation, highlighting HFL-AD's superior adaptability and scalability compared to these solutions.

*3) Alternative Upper-Layer Aggregation Algorithm:* In all the aforementioned experiments, we leverage FedAvg as the upper-layer aggregation algorithm in HFL-AD. However, the design of HFL-AD is not dependent on a specific upper-layer aggregation algorithm. To validate this independence, we replaced FedAvg with FedLAW [37] as an alternative upper-layer aggregation algorithm within HFL-AD and conducted repeated experiments. Fig. 9 shows the results on the CICDDoS2019 dataset. It is evident that HFL-AD is adaptable to different upper-layer aggregation algorithms, further showcasing HFL-AD's exceptional scalability.

### E. Practical Implications

The experimental results demonstrate that HFL-AD effectively mitigates the impact of contaminated training data in federated DDoS detection while maintaining scalability across different network environments and anomaly detection tasks. These findings have several real-world implications:

*1) Robust Federated Learning for Cybersecurity:* HFL-AD enables organizations (e.g., ISPs, cloud providers, or enterprise networks) to collaboratively train DDoS detection models without sharing raw traffic data, preserving privacy while improving threat detection. In addition, the dynamic threshold mechanism ensures resilience against evolving traffic patterns, making it suitable for real-world deployments where normal network behaviors frequently change.

*2) Adaptability to Diverse Threat Scenarios:* Unlike prior methods that struggle with low contamination ratios (1-8%), HFL-AD maintains high AUC and F1-scores even when malicious clients inject subtle adversarial updates. This makes it practical for real-world FL deployments, where attackers may attempt to evade detection by introducing small perturbations. The framework's effectiveness on non-traffic data (e.g., Census dataset) suggests broader applicability beyond DDoS, such as fraud detection, IoT anomaly detection, or industrial sensor monitoring.

*3) Scalability for Large-Scale Deployments:* HFL-AD supports hundreds of clients without significant performance degradation, making it feasible for distributed threat intelligence sharing among multiple organizations. The compatibility with different upper-layer aggregation algorithms (e.g., FedLAW) allows integration into existing FL frameworks, reducing adoption barriers.

## VIII. Discussion

HFL-AD detects DDoS attack traffic using hierarchical federated learning (FL). It operates in a semi-supervised manner with limited labeled anomaly samples. Specifically, HFL-AD addresses the following three problems in the real world: (1) Data contamination exists in monitored traffic, which can significantly impact model performance. It is impractical to examine all collected traffic samples due to the immense labor involved. (2) Traffic collected from a single host lacks

TABLE VI
OVERHEAD OF HFL-AD AND BASELINE METHODS ACROSS DATASETS

| Dataset | Aggregation Algorithm | Training Duration /s | Transmission Load /MB |
|---|---|---|---|
| CICDDoS2019 | Krum | 275.78 | 4,368 |
| | Trimmed Mean | 424.15 | 4,368 |
| | RFA | 702.62 | 4,368 |
| | FedAvg | 150.08 | 4,368 |
| | FLTrust | 307.24 | 4,368 |
| | HFL-AD | 339.12 | 4,368.13 |
| CICIoT2023 | Krum | 1,133.92 | 11,970 |
| | Trimmed Mean | 1,588.86 | 11,970 |
| | RFA | 2,669.89 | 11,970 |
| | FedAvg | 580.89 | 11,970 |
| | FLTrust | 1,274.98 | 11,970 |
| | HFL-AD | 1,346 | 11,970.24 |
| CICIoMT2024 | Krum | 682.57 | 6,300 |
| | Trimmed Mean | 917.99 | 6,300 |
| | RFA | 1,568.2 | 6,300 |
| | FedAvg | 328.35 | 6,300 |
| | FLTrust | 702.76 | 6,300 |
| | HFL-AD | 789.64 | 6,300.16 |

diversity and is insufficient for DDoS detection. Model training requires traffic data from various hosts. (3) Traffic collected from different hosts may contain private data that cannot be shared directly. Below, we discuss further aspects of HFL-AD in terms of its overhead, convergence, and robustness.

### A. Overhead

We evaluate the overhead of HFL-AD from the perspective of computation and communication, measured in terms of training duration and transmission load, respectively. Table VI compares the overhead of HFL-AD and baseline algorithms across three datasets. The results exhibit consistent trends, and we analyze the CICDDoS2019 dataset as a representative case.

For computation cost, FedAvg is the most efficient due to its non-robust averaging. Krum and FLTrust introduce moderate overhead from pairwise distance calculations and gradient normalization, respectively. Trimmed Mean and RFA incur higher costs due to coordinate-wise filtering and iterative geometric median computation. HFL-AD strikes a balance, being only 17.2% slower than FLTrust and 18.4% slower than Krum, while significantly outperforming them in robustness (Section VII-C). Similar trends hold for CICIoT2023 and CICIoMT2024, with RFA consistently being the slowest. In practical deployments, upper-layer clients in HFL-AD incur moderate computational overhead due to their dual role in local model training and contamination detection. Based on our experiments, the per-round computational time for an upper-layer client (e.g., a mid-tier IoT gateway with 4 CPU cores and 8 GB RAM) averages 2—3 seconds for local training and aggregation (100 epochs, batch size 32) and 15—38 seconds for contamination detection (dynamic thresholding and cosine similarity calculations). This totals 17—41 seconds per round, which is feasible for resource-constrained edge devices.

Regarding communication cost, the hierarchical structure of HFL-AD alleviates server-side bottlenecks while adding negligible overhead. The 0.003~0.004% (equivalent to 0.13~0.24 MB) increase stems from scalar cosine similarity exchanges among upper-layer clients, preserving bandwidth efficiency.

The moderate computational cost and minimal communication overhead are justified by HFL-AD's superior accuracy (Section VII-C), enabled by dynamic thresholding for contamination detection, attack traffic repurposing as signal, contamination-aware training, and multi-layer hierarchical defense.

### B. Feasibility of Labeled Anomaly Data

While labeled anomalies ($D_A$) are required for contamination detection, their acquisition is realistically manageable for the following reasons: i) Minimal data requirements: $D_A$ is small (e.g., 30 samples per upper-layer client) and can be sourced from historical attack traces (e.g., past intrusion logs, synthetic attack replay), public threat intelligence (e.g., CICIDS, CIC-DDoS datasets), and lightweight manual labeling (e.g., flagging anomalies during initial deployment). ii) One-time effort for long-term robustness: Unlike FLTrust (which requires continuous clean data updates), $D_A$ is static—once collected, it can be reused across rounds. Furthermore, attack patterns (e.g., DDoS signatures) exhibit temporal stability, reducing the need for frequent relabeling. iii) Hierarchical delegation lowers burden: Only upper-layer clients (a small subset of the network) need $D_A$, not every participant. In enterprise settings (e.g., IoT networks), these clients can be operated by administrators with access to security tools.

For heterogeneous scenarios where labeled anomalies are hard to obtain uniformly, we propose: i) Transferable attack representations: $D_A$ can include generalizable attack features (e.g., packet size anomalies in DDoS) rather than client-specific samples. Our experiments (Section VII) show robustness across multiple datasets (CICDDoS2019, CICIoT2023), suggesting $D_A$ need not be exhaustive. ii) Federated anomaly sharing: Upper-layer clients can securely share curated $D_A$ (e.g., via differential privacy) to mitigate data scarcity. iii) Fallback mechanisms: If $D_A$ is unavailable, HFL-AD can default to weaker baselines (e.g., Krum), though with reduced contamination resistance.

### C. Robustness

To evaluate the robustness of HFL-AD in more complex and varied environments, we have conducted additional experiments on the CICIoT2023 dataset, a large-scale IoT dataset designed for robust evaluation in complex attack scenarios. This dataset includes 33 attack types (e.g., DDoS, DoS, Recon, Spoofing, Mirai) executed in a realistic IoT topology with 105 devices, providing a highly challenging environment for flow-based anomaly detection.

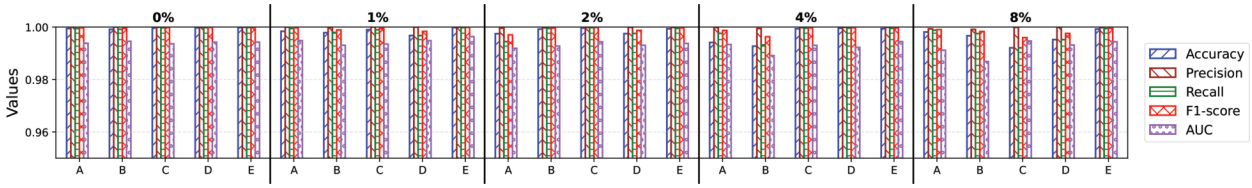



Fig. 10. Performance of HFL-AD across different contamination ratios under varying configurations. Labels A-E on the X-axis represent five distinct configurations. For each configuration, we evaluated different data samples at various contamination ratios, resulting in minor AUC value fluctuations.

TABLE VII
KEY ENHANCEMENTS IN JOURNAL VERSION VS. PRIOR CONFERENCE VERSION [39]

| Aspect | Prior Work [39] (Conference) | Journal Version | Significance of Enhancement |
|---|---|---|---|
| **Design** | Introduced a basic hierarchical FL framework for DDoS detection. | **Extended to a generalizable framework** for federated anomaly detection beyond DDoS (e.g., Census dataset). Added robustness analysis. | Broadens applicability to diverse anomaly detection tasks and network conditions. |
| **Algorithm** | Trained two local models in each upper-layer client per round to filter contaminated updates. | **Only one local model trained** in each upper-layer client per round (Algorithms 1–2) using attack traffic as supplementary data. | Improves effectiveness under evolving model behavior and varying contamination ratios (0–8%). |
| **Evaluation** | Evaluated on traffic datasets with limited metrics (AUC/F1). | **Comprehensive evaluation** on traffic datasets + non-traffic (Census). Added more SOTA baselines (e.g., RFA), scalability tests (client scalability, alternative aggregation algorithms) and threshold dynamics analysis (Figs. 6–9). | Demonstrates robustness across datasets (Table 3), scales to 100+ clients, and outperforms 5 SOTA baselines. |
| **Theoretical Analysis** | None. | **Convergence and overhead analysis** (Section VIII) with empirical validation of computational/communication costs. | Provides theoretical grounding for practical deployment. |
| **Reproducibility** | Minimal details on hyperparameters. | **Full experimental settings** (Tables 1–2), code, and dataset links provided. | Ensures reproducibility and adoption. |

Specifically, we designed five distinct configurations with varying compositions of DDoS and non-DDoS attacks, including:

1) Configuration A: 4 DDoS attacks (ICMP/UDP/TCP/SY-N Flood).
2) Configuration B: 3 DDoS attacks + 1 non-DDoS (DNS Spoofing).
3) Configuration C: 2 DDoS attacks + 2 non-DDoS (Vulnerability Scan, DNS Spoofing).
4) Configuration D: 1 DDoS attack + 3 non-DDoS (Browser Hijacking, Vulnerability Scan, DNS Spoofing).
5) Configuration E: 4 non-DDoS attacks (Dictionary Brute Force, Browser Hijacking, etc.).

For each configuration, attack samples were injected into normal traffic at multiple contamination ratios to simulate varying interference intensities. In addition, within each configuration, all attack types were equally represented to ensure unbiased evaluation of our method's resilience against diverse attack interactions. The results (as shown in Fig. 10) demonstrate that HFL-AD maintains high detection performance across all configurations, even under aggressive mixed-attack scenarios. This comprehensive validation underscores HFL-AD's adaptability to heterogeneous, real-world IoT threats.

## IX. CONCLUSION AND FUTURE WORK

In this paper, we proposed HFL-AD, a hierarchical federated learning framework for DDoS detection that mitigates data contamination while preserving privacy and improving data diversity. HFL-AD designates clients with supplemental labeled anomaly data as upper-layer clients, leveraging gradient cosine similarity to filter contaminated updates. Experiments demonstrate HFL-AD's superiority over SOTA methods in detection accuracy, scalability, and adaptability to non-traffic tasks.

**Practical Deployment Scenarios.** HFL-AD's robustness and privacy-preserving design make it particularly valuable for real-world collaborations:

- **ISP Networks:** Multiple ISPs can jointly train DDoS detectors on traffic from geographically distributed networks without sharing raw data, enabling early threat mitigation while complying with data sovereignty regulations.
- **Industrial IoT:** In smart factories, HFL-AD can aggregate anomaly reports from sensor fleets (e.g., predictive maintenance alerts) across different manufacturers, improving detection of adversarial tampering or equipment failures.
- **Cloud Providers:** Federated threat intelligence sharing among cloud platforms (e.g., AWS, Azure) could identify cross-tenant attack patterns while isolating sensitive customer data.

Future work should address dynamic client churn and reduce reliance on labeled upper-layer data, further bridging research and deployment gaps.

## APPENDIX A
## COMPARISON TO CONFERENCE VERSION

Table VII explicitly highlights the novel advancements of this work in design, algorithm, and evaluation compared to the conference version.

Specifically, we discuss the novel contributions of this work over the conference version from four aspects.

1) **Algorithmic Innovation.**
   - **Aggregation Algorithm:** The conference version required training two local models in each upper-layer client for dynamic threshold calculation. In this journal version, we propose training only one local model per upper-layer client, which proves more effective (Figs. 5–6).
2) **Generalizability.**
   - Extended from DDoS-only detection to **multi-scenario anomaly detection** (e.g., Census income data in Section VII-D).
   - Support for **alternative upper-layer aggregators** (FedLAW in Fig. 9) and scalability to 25 lower-layer clients per group (Fig. 8).
3) **Enhanced Evaluation.**
   - **Additional Metrics:** Introduced more evaluation metrics (e.g., accuracy, precision, recall, and F1-score) to comprehensively assess HFL-AD's performance (Fig. 6).
   - **Comparative Baselines:** Added RFA [34] and FedLAW [37] to validate superiority over newer FL methods.
4) **Practical Implications.**
   - Added **overhead analysis** (Section VIII) showing only 32% runtime increase versus FedAvg despite dynamic thresholding.
   - **Real-world applicability** discussion (Section VII-E) for ISPs and IoT networks.

## ACKNOWLEDGMENT

The authors would like to thank the anonymous reviewers for their valuable feedback on this paper.

## REFERENCES

[1] B. Bala and S. Behal, "AI techniques for IoT-based DDoS attack detection: Taxonomies, comprehensive review and research challenges," *Comput. Sci. Rev.*, vol. 52, May 2024, Art. no. 100631.

[2] S. Ranger. (Feb. 28, 2018). *GitHub Hit With the Largest DDoS Attack Ever Seen*. ZDNet. [Online]. Available: https://goo.gl/BmqekG

[3] J. Yan, H. Zhou, and W. Wang, "Intelligent network element: A programmable switch based on machine learning to defend against DDoS attacks," *Inf. Syst. Frontiers*, vol. 2025, pp. 1–20, Jan. 2025.

[4] V. Pourahmadi, H. A. Alameddine, M. A. Salahuddin, and R. Boutaba, "Spotting anomalies at the edge: Outlier exposure-based cross-silo federated learning for DDoS detection," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 5, pp. 4002–4015, Sep. 2023.

[5] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2018, pp. 1–15.

[6] S. Wu, J. Zhao, and G. Tian, "Understanding and mitigating data contamination in deep anomaly detection: A kernel-based approach," in *Proc. 31st Int. Joint Conf. Artif. Intell.*, Jul. 2022, pp. 2319–2325.

[7] Z. Wang, H. Xu, J. Liu, Y. Xu, H. Huang, and Y. Zhao, "Accelerating federated learning with cluster construction and hierarchical aggregation," *IEEE Trans. Mobile Comput.*, vol. 22, no. 7, pp. 3805–3822, Jul. 2022.

[8] W. Y. B. Lim, J. S. Ng, Z. Xiong, D. Niyato, C. Miao, and D. I. Kim, "Dynamic edge association and resource allocation in self-organizing hierarchical federated learning networks," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 12, pp. 3640–3653, Dec. 2021.

[9] P. Blanchard, E. M. E. Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, Dec. 2017, pp. 118–128.

[10] D. Yin, Y. Chen, R. Kannan, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proc. Int. Conf. Mach. Learn.*, Jul. 2018, pp. 5650–5659.

[11] X. Cao, M. Fang, J. Liu, and N. Z. Gong, "FLTrust: Byzantine-robust federated learning via trust bootstrapping," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2021, pp. 1–18.

[12] L. Ruff et al., "Deep semi-supervised anomaly detection," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Jan. 2019, pp. 1–23.

[13] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Int. Conf. Artif. Intell. Stat. (AISTATS)*, Jan. 2016, pp. 1273–1282.

[14] C. Qiu, T. Pfrommer, M. Kloft, S. Mandt, and M. Rudolph, "Neural transformation learning for deep anomaly detection beyond images," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2021, pp. 8703–8714.

[15] C. Qiu, A. Li, M. Kloft, M. Rudolph, and S. Mandt, "Latent outlier exposure for anomaly detection with contaminated data," in *Proc. Int. Conf. Mach. Learn. (ICML)*, Jan. 2022, pp. 18153–18167.

[16] M. Fang, X. Cao, J. Jia, and N. Gong, "Local model poisoning attacks to Byzantine-robust federated learning," in *Proc. 29th USENIX Secur. Symp.*, 2020, pp. 1605–1622.

[17] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, "Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy," in *Proc. Int. Carnahan Conf. Security Technol. (ICCST)*, 2019, pp. 1–8, doi: 10.1109/CCST.2019.8888419.

[18] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," *Sensors*, vol. 23, no. 13, p. 5941, Jun. 2023, doi: 10.3390/s23135941.

[19] S. Dadkhah, E. C. P. Neto, R. Ferreira, R. C. Molokwu, S. Sadeghi, and A. A. Ghorbani, "CICIoMT2024: Attack vectors in healthcare devices—A multi-protocol dataset for assessing IoMT device security," *Internet Things*, vol. 28, pp. 1–29, Dec. 2024, doi: 10.20944/preprints202402.0898.v1.

[20] J. G. Almaraz-Rivera, J. A. Perez-Diaz, and J. A. Cantoral-Ceballos, "Transport and application layer DDoS attacks detection to IoT devices by using machine learning and deep learning models," *Sensors*, vol. 22, no. 9, p. 3367, Apr. 2022.

[21] V. Mothukuri, R. M. Parizi, S. Pouriyeh, Y. Huang, A. Dehghantanha, and G. Srivastava, "A survey on security and privacy of federated learning," *Future Gener. Comput. Syst.*, vol. 115, pp. 619–640, Oct. 2020.

[22] R. Benameur, A. Dahane, S. Souihi, and A. Mellouk, "A novel federated learning based intrusion detection system for IoT networks," in *Proc. IEEE Int. Conf. Commun.*, Jun. 2024, pp. 2402–2407.

[23] K. Yang, J. Zhang, Y. Xu, and J. Chao, "DDoS attacks detection with AutoEncoder," in *Proc. NOMS-IEEE/IFIP Netw. Oper. Manage. Symp.*, Apr. 2020, pp. 1–9.

[24] A. Zainudin, L. A. C. Ahakonye, R. Akter, D.-S. Kim, and J.-M. Lee, "An efficient hybrid-DNN for DDoS detection and classification in software-defined IIoT networks," *IEEE Internet Things J.*, vol. 10, no. 10, pp. 8491–8504, May 2023.

[25] D. Akgun, S. Hizal, and U. Cavusoglu, "A new DDoS attacks intrusion detection model based on deep learning for cybersecurity," *Comput. Secur.*, vol. 118, Jul. 2022, Art. no. 102748.

[26] C. Xu et al., "A federated learning architecture for blockchain DDoS attacks detection," *IEEE Trans. Services Comput.*, vol. 17, no. 5, pp. 1911–1923, Sep. 2024.

[27] J. Mateus, G.-A.-L. Zodi, and A. Bagula, "Federated learning-based solution for DDoS detection in SDN," in *Proc. Int. Conf. Comput., Netw. Commun. (ICNC)*, Big Island, HI, USA, Feb. 2024, pp. 875–880.

[28] R. Doriguzzi-Corin and D. Siracusa, "FLAD: Adaptive federated learning for DDoS attack detection," *Comput. Secur.*, vol. 137, Feb. 2024, Art. no. 103597.

[29] M. Kim, J. Yu, J. Kim, T.-H. Oh, and J. K. Choi, "An iterative method for unsupervised robust anomaly detection under data contamination," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 10, pp. 13327–13339, Oct. 2024.

[30] Q. Su, B. Tian, H. Wan, and J. Yin, "Anomaly detection under contaminated data with contamination-immune bidirectional GANs," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 11, pp. 5605–5620, Nov. 2024.

[31] H. Zhang, L. Cao, P. VanNostrand, S. Madden, and E. A. Rundensteiner, "ELITE: Robust deep anomaly detection with meta gradient," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2021, pp. 2174–2182.

[32] J. Xu, H. Fan, Q. Wang, Y. Jiang, and Q. Duan, "Adaptive idle model fusion in hierarchical federated learning for unbalanced edge regions," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 5, pp. 4603–4616, Sep. 2024.

[33] T. Zhang, K.-Y. Lam, and J. Zhao, "Device scheduling and assignment in hierarchical federated learning for Internet of Things," *IEEE Internet Things J.*, vol. 11, no. 10, pp. 18449–18462, May 2024.

[34] K. Pillutla, S. M. Kakade, and Z. Harchaoui, "Robust aggregation for federated learning," *IEEE Trans. Signal Process.*, vol. 70, pp. 1142–1154, 2022.

[35] Y. Dong et al., "HorusEye: A realtime IoT malicious traffic detection framework using programmable switches," in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 571–588.

[36] J. Yin et al., "Masked memory network for semi-supervised anomaly detection in Internet of Things," *IEEE Internet Things J.*, vol. 11, no. 19, pp. 30636–30647, Oct. 2024.

[37] Z. Li, T. Lin, X. Shang, and C. Wu, "Revisiting weighted aggregation in federated learning with neural networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, Jan. 2023, pp. 19767–19788.

[38] M. Arjovsky, S. Chintala, and L. Bottou, "Wasserstein generative adversarial networks," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 214–223.

[39] H. Huang, J. Gui, J. Hong, and C. Hua, "HFL-AD: A hierarchical federated learning framework for solving data contamination in DDoS detection," in *Proc. IEEE 23rd Int. Conf. Trust, Secur. Privacy Comput. Commun. (TrustCom)*, Dec. 2024, pp. 2376–2381.

[40] Z. Lin et al., "Hierarchical split federated learning: Convergence analysis and system optimization," *IEEE Trans. Mobile Comput.*, early access, Apr. 29, 2025, doi: 10.1109/TMC.2025.3565509.

[41] J. Liu, X. Wei, X. Liu, H. Gao, and Y. Wang, "Group-based hierarchical federated learning: Convergence, group formation, and sampling," in *Proc. 52nd Int. Conf. Parallel Process.*, Aug. 2023, pp. 264–273.

[42] L. Liu, J. Zhang, S. Song, and K. B. Letaief, "Hierarchical federated learning with quantization: Convergence analysis and system design," *IEEE Trans. Wireless Commun.*, vol. 22, no. 1, pp. 2–18, Jan. 2023.

[43] Y. Deng et al., "A communication-efficient hierarchical federated learning framework via shaping data distribution at edge," *IEEE/ACM Trans. Netw.*, vol. 32, no. 3, pp. 2600–2615, Jun. 2024.

[44] H. Lu et al., "FL-AMM: Federated learning augmented map matching with heterogeneous cellular moving trajectories," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 12, pp. 3878–3892, Dec. 2023.

[45] E. Owusu et al., "Online network DoS/DDoS detection: Sampling, change point detection, and machine learning methods," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 1, pp. 15–35, 1st Quart., 2024.

[46] Y. S. N. Fotse, V. K. Tchendji, and M. Velempini, "Federated learning based DDoS attacks detection in large scale software-defined network," *IEEE Trans. Comput.*, vol. 74, no. 1, pp. 101–115, Jan. 2025.

[47] S. P. Sanon, R. Reddy, C. Lipps, and H. D. Schotten, "DDoS attacks in communication: Analysis and mitigation of unreliable clients in federated learning," in *Proc. IEEE 21st Consum. Commun. Netw. Conf. (CCNC)*, Jan. 2024, pp. 986–989.

[48] UCI Mach. Learn. Repository. (1997). *Census-Income (KDD) Dataset*. [Online]. Available: https://archive.ics.uci.edu/ml/datasets/Census+Income